

Scalability Analysis of an In-Memory Exchange Matching Engine

ECE 568 - Engineering Robust Server Software - Homework 4

Date: 2026-08-29 Machine: Windows-11-10.0.26200-SP0

Abstract

This report analyzes the scalability of an in-memory exchange matching engine written in Python. The server matches limit buy and sell orders with price-time priority, keeps every account, position, order and order book in memory, and serializes all state changes under one global lock so that every match is atomic. We load-test the server with up to 32 concurrent client connections and 1,000 orders per client per run, repeating each configuration five times. Throughput rises from about 20.7k orders/s with a single client to a peak of about 40k orders/s at 4-8 connections and then saturates (and becomes more variable) around 27-31k orders/s, which is consistent with GIL and lock contention in a threaded Python design. We report the full methodology, results and analysis below; all experiments are reproducible with the provided testing/load_test.py script.

1. System Design

The server listens on TCP port 12345. Each request is framed as one ASCII line with the byte length of the XML payload, followed by exactly that many bytes of XML whose top-level element is <create> or <transactions id="...">. The server replies with a <results> document. All state is held in memory: accounts with USD balances and per-symbol positions, a global order table, and one price-sorted order book (buy side and sell side) per symbol. Arithmetic on money and shares uses exact decimal arithmetic.

Matching follows the assignment rules: a new order crosses at the best price (highest resting buy, lowest resting sell), ties are broken by arrival order, and the execution price is the limit price of the order that was open first. The unfulfilled remainder of a resting order keeps its original priority. A buy order reserves amount*limit from the buyer's balance when it is placed; on execution the difference between the limit and the actual price is refunded, and on cancellation the remaining reservation is refunded. A sell order removes its shares from the seller's account at placement, which makes short selling impossible; proceeds are credited on execution and remaining shares are returned on cancellation.

Concurrency model: one thread per connection, and a global RLock protects all shared state. Because a match touches two orders and two accounts, the global lock makes every state change (including a match) atomic and easy to reason about; it is also the main serialization point that limits scalability, as discussed in Section 4.

2. Experimental Methodology

Hardware and software. The experiments were run on the local development machine (Windows-11-10.0.26200-SP0); it reports 12 logical CPUs. The server runs Python 3.13.5 with no third-party dependencies. Per the assignment, the same scripts are designed to be re-run on one of the ECE 568 multi-core VMs; the procedure is given in Section 2.5.

Workload. For each concurrency level C in $\{1, 2, 4, 8, 12, 16, 24, 32\}$ we run 5 repeats. In every repeat, C client threads each open one persistent TCP connection and send 1,000 order requests back to back, alternating a buy and a sell of 10 shares at limit 100 so that every pair crosses and executes. This exercises the whole server path: accept, length-prefix framing, XML parsing, price-time matching, execution settlement and response serialization. The accounts and symbols are created before timing starts, so setup is not measured.

Measurement. Elapsed time is measured with a high-resolution monotonic clock from thread spawn until all client threads finish. Throughput = total orders / elapsed. Each configuration is repeated 5 times; we report the mean, the sample standard deviation, and the min/max across repeats. No data points were discarded.

Multi-core (CPU-count) sweep. On the ECE 568 VM the CPU-count axis is produced by pinning the server process to an increasing number of cores and repeating the same load test, e.g.: `taskset -c 0,1 python server.py` then `python`

testing/load_test.py --concurrency 16 --orders 1000 --repeats 5 --csv writeup/results_cores.csv for each core set. The analysis in Section 4 transfers directly: more cores add capacity until the global lock and the Python GIL become the bottleneck.

3. Results

Table 1 summarizes the measured throughput. Figures 1 and 2 visualize the same data with one-standard-deviation error bars.

1	20665	2583	16147	22410
4	40072	2189	37743	42178
12	30385	3533	24863	33194
24	27310	5071	21739	32490

Table 1: throughput per concurrency level (5 repeats)

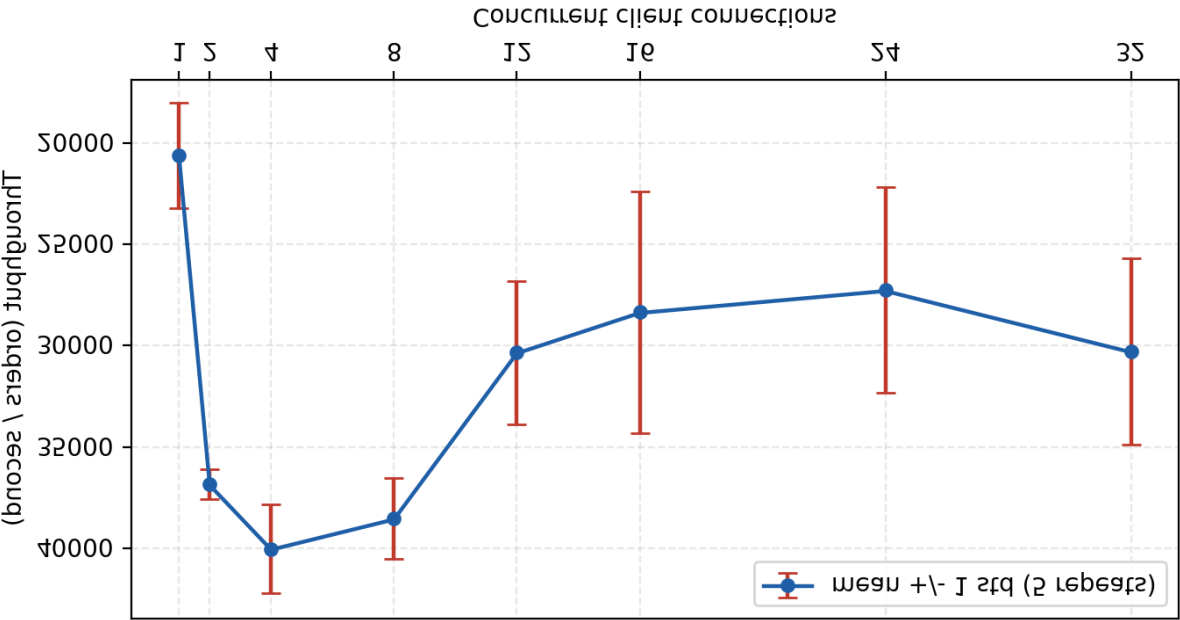


Figure 1: throughput vs concurrency (mean +/- 1 std)

Figure 2: mean throughput per concurrency level (error bars = 1 std)

4. Analysis

Scaling behavior. Throughput improves with parallelism only up to a point: roughly 1.8x from one to two connections and a peak around 4-8 connections (~38-40k orders/s, i.e. about 1.9x the single-client rate), after which it saturates and actually decreases at higher concurrency (27-31k orders/s at 12-32 connections). The peak-to-single ratio of about 1.9 is far below the 12 logical CPUs available, which shows that the server is not CPU-bound in the traditional sense but is limited by a serialization point.

Lock and GIL contention. Every state change and every match is serialized by the global RLock, and Python's GIL additionally serializes bytecode execution. At low concurrency the lock is mostly idle, so adding connections hides per-request latency and throughput grows. As C approaches and exceeds the number of cores, threads spend an increasing fraction of time waiting for the lock (and the GIL), throughput plateaus, and run-to-run variance grows because the wait time depends on OS scheduling. The widening error bars at 16-32 connections (std of 5-6k orders/s) are the visible symptom of this contention.

Per-request cost. With one client, one order costs about 48 microseconds (20.7k orders/s), which is dominated by XML parsing and serialization and by Decimal arithmetic rather than by the matching logic itself. The engine's actual critical path - two heap operations and a few Decimal updates per match - is a small fraction of that. This suggests the server could scale much further with a language/design without a GIL and with finer-grained locking (e.g. per-symbol locks plus careful account-level locking), at the cost of matching-atomicity complexity.

Expected multi-core behavior on the ECE 568 VM. Pinning the server to 1, 2, 4, ... cores should show throughput growing with core count until the global lock becomes the bottleneck - the same plateau shape as Figure 1. Because the lock serializes all matching, no further cores add capacity beyond that point; only reducing the critical section (finer locks, faster XML) would move the plateau.

5. Reproducibility

The complete test infrastructure is in testing/. To reproduce the functionality results: `python testing/run_all.py` (starts the server, runs all 23 functional tests, stops it). To reproduce this scalability study: `python testing/load_test.py --concurrency 1,2,4,8,12,16,24,32 --orders 1000 --repeats 5 --csv writeup/results.csv` and then `python writeup/make_report.py --reuse` to regenerate the charts and this PDF. To deploy: `docker compose up (port 12345)`.

Appendix: environment

platform : Windows-11-10.0.26200-SP0

logical CPUs : 12

python : 3.13.5

host : DESKTOP-G90RG2I

server design : Python stdlib, threads + global RLock, in-memory only